Day 3 exercises

Day 3 Exercises: Development Tools & Workflows

PhD Course in Integrative Neurosciences - Introduction to Scientific Programming

Overview

This document contains hands-on exercises to practice the development tools and workflows covered in the course. Complete these exercises in order, as later exercises build on earlier ones. Note that this is very comprehensive and is meant to make you familiar and proficient with the contents discussed. It is highly advisable that you try to do as much as possible, but you are not required to complete, nor do you strictly need this to understand the remainder of the course.

Table of Contents

1. Jupyter Notebooks Exercises
2. Virtual Environments Exercises
3. Git Fundamentals Exercises
4. GitHub Workflows Exercises
5. Integrated Workflow Project

Exercise 1: Jupyter Notebooks

Learning Objectives:

- Create and navigate Jupyter notebooks
- Use Markdown for documentation
- Execute code cells and understand kernel state
- Generate and summarise data with plain Python and display results as text and rich output
- Use magic commands

Part A: Your First Notebook

1. **Launch JupyterLab**

```
jupyter lab
```

2. **Create a new notebook**
   - Name it `01_introduction.ipynb`
3. **Add a title cell** (Markdown):

```
# My First Scientific Notebook
## Introduction to Data Analysis

**Author**: Your Name
**Date**: YYYY-MM-DD

This notebook demonstrates basic data analysis techniques.
```

4. **Import modules** (Code cell):

```
import math
import random
import statistics
```

5. **Test cell execution order**:

   In cell 1:

```
x = 10
print(f"x is set to {x}")
```

   In cell 2:

```
print(f"x is still {x}")
```

   In cell 3:

```
x = 20
print(f"x is now {x}")
```

   Execute cells in order: 1, 2, 3, then re-execute cell 2. What happens?

6. **Add an explanation** (Markdown):

```
## Understanding Kernel State

The kernel maintains variables across cells. When we changed `x`
to 20,
re-running cell 2 showed the new value because the kernel
remembers
the latest value.

**Key insight**: Cell order in the notebook file doesn't matter—
execution order does!
```

Part B: Data Analysis Practice

Continue in the same notebook. Everything in this part uses only plain Python and the standard library.

7. **Add a section header** (Markdown):

```
## Data Generation and Analysis
```

8. **Generate sample data** (Code):

```
# Set random seed for reproducibility
random.seed(42)

# Generate sample data: temperature measurements for 30 days
days = list(range(1, 31))
temperature = [
    round(20 + 5 * math.sin(day / 5) + random.gauss(0, 1), 1)
    for day in days
]

print(f"Generated {len(days)} temperature measurements")
print(f"Temperature range: {min(temperature):.1f}°C to
{max(temperature):.1f}°C")
```

9. **Basic statistics** (Code):

```
mean_temp = statistics.mean(temperature)
median_temp = statistics.median(temperature)
std_temp = statistics.stdev(temperature)

print(f"Mean temperature: {mean_temp:.2f}°C")
print(f"Median temperature: {median_temp:.2f}°C")
print(f"Standard deviation: {std_temp:.2f}°C")
```

10. **Sort and filter** (Code):

```
# Five warmest days
records = list(zip(days, temperature))
warmest = sorted(records, key=lambda record: record[1],
reverse=True)[:5]
print("Five warmest days:")
for day, temp in warmest:
    print(f"  day {day:2d}: {temp:.1f}°C")

# Days that are more than one standard deviation above the mean
hot_days = [day for day, temp in records if temp > mean_temp +
std_temp]
print(f"Days above mean + 1 std dev: {hot_days}")
```

11. **Print a table and a text bar chart** (Code):

```
print(f"{'Day':>4} | {'Temp (°C)':>9} | Bar")
print("-" * 50)
for day, temp in records:
    bar = '#' * round(temp - 10)   # one '#' per degree above 10°C
    print(f"{day:>4} | {temp:>9.1f} | {bar}")

print(f"\nMean: {mean_temp:.1f}°C (±{std_temp:.1f}°C)")
```

12. **Document findings** (Markdown):

```
## Results

The temperature data shows:
- A slow, wave-like pattern (see the bars in the table)
- Random fluctuations around the pattern (±1-2°C)
- Mean temperature of approximately 20°C

This pattern could represent:
- Daily temperature cycles
- Weekly weather patterns
- Measurement noise superimposed on a trend
```

Part C: Magic Commands

13. **Add a section** (Markdown):

```
## Exploring Magic Commands
```

14. **Time a computation**:

```
# Time a single statement
%timeit sum(range(10000))
```

Time a whole code block (the `%%` cell magic must be the **first line** of the cell):

```
%%timeit
total = 0
for i in range(10000):
    total += i
```

Time a cell once (instead of many repetitions):

```
%%time
squares = [i ** 2 for i in range(1_000_000)]
```

15. **View variables**:

```
%who

%whos
```

16. **Save variables**:

```
# Save data for use in other notebooks
%store temperature
%store days

# List stored variables
%store
```

17. **Run a Python file from the notebook**:

```
%%writefile helper.py
def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32

print("helper.py was executed")
print(celsius_to_fahrenheit(20))

%run helper.py
```

After `%run`, the functions defined in the file are available in the notebook:

```
celsius_to_fahrenheit(37)
```

18. **System commands**:

```
# List current directory (Unix/Mac)
!ls -la

# Or on Windows
!dir

%%bash
echo "This cell runs in bash"
python --version
```

List all available magics with `%lsmagic`.

Part D: Rich Display

Notebooks can show more than plain text. The `IPython.display` module (already available in your environment) lets you render Markdown, HTML and formulas from code.

19. **Display Markdown from code**:

```
from IPython.display import display, Markdown, HTML, Math, Latex

display(Markdown(f"""
### Summary
- **Mean**: {mean_temp:.2f} °C
- **Standard deviation**: {std_temp:.2f} °C
- **Number of days**: {len(days)}
"""))
```

20. **Display a formula**:

```
        display(Math(r"\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i"))
        display(Latex(r"The sample standard deviation is $s =
\sqrt{\frac{1}{n-1}\sum (x_i - \bar{x})^2}$"))
```

21. **Build an HTML table with a function**:

```
        def html_table(rows, headers):
            """Return an HTML table (as a string) for a list of rows."""
            html = ['<table>', '<tr>']
            html += [f'<th>{h}</th>' for h in headers]
            html.append('</tr>')
            for row in rows:
                html.append('<tr>')
                for value in row:
                    text = f'{value:.1f}' if isinstance(value, float) else
value
                    html.append(f'<td>{text}</td>')
                html.append('</tr>')
            html.append('</table>')
            return ''.join(html)

        display(HTML(html_table(warmest, ['Day', 'Temperature (°C)'])))
```

22. **Colour the output** (Code):

```
        def coloured_value(value, threshold):
            colour = 'red' if value > threshold else 'steelblue'
            return f'<span style="color: {colour}; font-weight: bold">
{value:.1f}</span>'

        cells = ' '.join(coloured_value(t, mean_temp + std_temp) for t in
temperature)
        display(HTML(f"<p>Values above mean + 1 std dev are red:</p><p>
{cells}</p>"))
```

Part E: Best Practices

23. **Clean up and prepare for sharing**:

- Click **Kernel → Restart Kernel and Clear Outputs of All Cells**
- Click **Kernel → Restart Kernel and Run All Cells**
- Check that all cells execute without errors
- Save the notebook: **Ctrl+S**

24. **Export notebook**:

- **File → Save and Export Notebook As → HTML**
- Open the HTML file in a browser to see the static version

Questions to Answer:

1. What happens to variables when you restart the kernel?

2. Why is it important to run "Restart & Run All" before sharing?
3. What's the difference between `%timeit` and `%%timeit`?
4. When should you use Markdown cells vs code cells with comments?

Exercise 2: Virtual Environments

Learning Objectives:

- Create and manage virtual environments
- Install and manage packages
- Pin package versions
- Export and recreate environments
- Compare venv and conda

In this exercise we use the small, popular package `requests` (an HTTP library) purely as an example of something to install. You do not need to use it for anything else.

Part A: Python venv

1. **Create a project directory**:

   ```
   mkdir data_project
   cd data_project
   ```

2. **Create a virtual environment**:

   ```
   python -m venv venv_data
   ```

   Question: What directories and files were created?

3. **Activate the environment**:

   Linux/Mac:

   ```
   source venv_data/bin/activate
   ```

   Windows:

   ```
   venv_data\Scripts\activate
   ```

   Your prompt should now show `(venv_data)`.

4. **Check Python location**:

   ```
   which python    # Linux/Mac
   where python    # Windows
   ```

   Question: Is this the system Python or the venv Python?

5. **Install a package**:

   ```
   pip install requests
   ```

6. **List installed packages**:

   ```
   pip list
   ```

   Question: How many packages are installed? (Note: dependencies are included)

7. **Create requirements file**:

   ```
   pip freeze > requirements.txt
   ```

   Open `requirements.txt` and examine the contents.

8. **Test the environment**:

   ```
   python -c "import requests; print(requests.__version__)"
   ```

9. **Deactivate**:

   ```
   deactivate
   ```

   Your prompt should return to normal.

10. **Try importing requests now**:

    ```
    python -c "import requests"
    ```

    Question: What happens? Why?

Part B: Reproducing an Environment

11. **Create a second environment**:

    ```
    python -m venv venv_test
    source venv_test/bin/activate  # or venv_test\Scripts\activate on Windows
    ```

12. **Install from requirements**:

    ```
    pip install -r requirements.txt
    ```

13. **Verify identical packages**:

    ```
    pip list
    ```

    Question: Is this list identical to the first environment?

14. **Test imports**:

    ```
    python -c "import requests; print('Success!', requests.__version__)"
    ```

15. **Pin a specific version** (still in `venv_test`):

    ```
    pip install requests==2.28.2
    python -c "import requests; print(requests.__version__)"
    ```

Now upgrade again and check the version:

```
pip install --upgrade requests
python -c "import requests; print(requests.__version__)"
```

Question: Which version do you have now? Why would you want to *pin* a version in a project?

16. **Clean up**:

```
deactivate
```

Part C: Conda Environments

17. **Verify conda installation**:

```
conda --version
```

If it is missing, go back to the Day 2 setup guide (`day02-fundamentals/setup/setup_tutorial.md`) and install Miniconda.

18. **Create conda environment**:

```
conda create -n analysis_env python=3.11
```

Type `y` when prompted.

19. **Activate environment**:

```
conda activate analysis_env
```

20. **Install packages with conda**:

```
conda install requests
```

21. **List installed packages**:

```
conda list
```

Question: Compare the output to `pip list`. What differences do you notice?

22. **Export environment**:

```
conda env export > environment.yml
```

Open `environment.yml` and examine the structure.

23. **Create environment from file**:

```
conda deactivate
conda env create -f environment.yml -n analysis_clone
conda activate analysis_clone
conda list
```

Question: Is the package list identical to `analysis_env`?

24. **Test different Python versions**:

```
conda deactivate
conda create -n py39 python=3.9
conda activate py39
python --version

conda create -n py311 python=3.11
conda activate py311
python --version
```

25. **List all environments**:

```
conda env list
```

26. **Clean up**:

```
conda deactivate
conda env remove -n analysis_clone
conda env remove -n py39
conda env remove -n py311
```

Part D: Comparison Exercise

27. **Create a curated `requirements.txt`**:

    Create a file named `requirements_minimal.txt`:

```
requests>=2.28.0
jupyterlab>=4.0.0
```

28. **Create environments with both tools**:

    venv:

```
python -m venv venv_minimal
source venv_minimal/bin/activate
pip install -r requirements_minimal.txt
pip list > venv_packages.txt
deactivate
```

    conda:

```
conda create -n conda_minimal python=3.11
conda activate conda_minimal
conda install requests jupyterlab
conda list > conda_packages.txt
conda deactivate
```

29. **Compare package lists**:

```
wc -l venv_packages.txt conda_packages.txt
```

    Question: Which has more packages? Why?

Part E: Real-World Scenario

30. **Scenario**: A colleague shares a project with this `environment.yml`. It pins an exact version of `requests` so that everybody gets the same behaviour:

    Create `project_environment.yml`:

    ```yaml
    name: shared_project
    channels:
      - conda-forge
      - defaults
    dependencies:
      - python=3.11
      - requests=2.31
      - jupyterlab
      - pip
    ```

31. **Set up the environment**:

    ```
    conda env create -f project_environment.yml
    conda activate shared_project
    ```

32. **Verify installation**:

    ```
    python -c "import requests; print(f'requests version: {requests.__version__}')"
    ```

    Question: Does the printed version match the one pinned in the file (2.31.x)?

33. **Two projects, two versions**: Another (older) project of yours needs a different version of `requests`. Create a `requirements.txt` for it using `venv`:

    ```
    conda deactivate
    mkdir old_project
    cd old_project
    python -m venv venv
    source venv/bin/activate        # or venv\Scripts\activate on Windows
    echo "requests==2.28.2" > requirements.txt
    pip install -r requirements.txt
    python -c "import requests; print(requests.__version__)"
    deactivate
    ```

    Question: Why can both versions exist on the same computer without conflict?

34. **Document the setup**:

    Create `SETUP.md`:

    ```
    # Environment Setup

    ## Prerequisites
    - Miniconda (see the Day 2 setup guide)
    ```

```
## Installation
1. Create environment:
   ```bash
   conda env create -f project_environment.yml
```

2. Activate environment:

```
conda activate shared_project
```

3. Verify installation:

```
python -c "import requests; print('Success!')"
```

Updating

To update the environment file after installing new packages:

```
conda env export > project_environment.yml
```

Questions to Answer:

1. When would you use `venv` vs `conda`?
2. What's the difference between `pip freeze` and a curated `requirements.txt`?
3. Why include version numbers in requirements files?
4. How do you handle platform-specific dependencies?
5. What's the purpose of the `channels` section in `environment.yml`?

Exercise 3: Git Fundamentals

Learning Objectives:

- Initialize repositories
- Make commits
- View history
- Use branches
- Merge changes
- Resolve conflicts

The code in this exercise is plain Python (functions like the ones you saw on Day 2). You are not asked to write it from scratch: **copy the code exactly as shown** and focus on the Git commands. The project is a tiny DNA sequence toolkit; no external packages are needed.

Part A: Basic Git Operations

1. **Configure Git** (if not already done):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```
```

```
git config --global init.defaultBranch main
```

2. **Create a new project**:

```
mkdir gene_analysis
cd gene_analysis
```

3. **Initialize Git repository**:

```
git init
git status
```

Question: What does `git status` tell you?

4. **Create a Python script**:

Create `sequence_analyzer.py`:

```
"""
DNA Sequence Analyzer
Simple tools for analyzing DNA sequences.
"""

def gc_content(sequence):
    """Calculate GC content of DNA sequence."""
    sequence = sequence.upper()
    gc_count = sequence.count('G') + sequence.count('C')
    return gc_count / len(sequence) * 100

if __name__ == "__main__":
    test_seq = "ATGCATGCATGC"
    print(f"GC content: {gc_content(test_seq):.1f}%")
```

5. **Check status**:

```
git status
```

Question: What is the file's status?

6. **Stage the file**:

```
git add sequence_analyzer.py
git status
```

Question: How did the status change?

7. **Make first commit**:

```
git commit -m "Add basic GC content calculator"
```

8. **View commit history**:

```
git log
git log --oneline
```

9. **Create README**:

   Create `README.md`:

   ```
   # Gene Analysis Tools

   Tools for analyzing DNA sequences.

   ## Features
   - GC content calculation

   ## Usage
   ```python
   from sequence_analyzer import gc_content

   seq = "ATGCATGC"
   gc = gc_content(seq)
   print(f"GC content: {gc:.1f}%")
   ```

10. **Stage and commit**:

    ```
    git add README.md
    git commit -m "Add README with usage instructions"
    ```

11. **Create .gitignore**:

    Create `.gitignore`:

    ```
    __pycache__/
    *.pyc
    .ipynb_checkpoints/
    *.egg-info/
    .venv/
    venv/
    data/
    results/
    ```

12. **Stage and commit**:

    ```
    git add .gitignore
    git commit -m "Add gitignore for Python project"
    ```

13. **View history**:

    ```
    git log --oneline --graph
    ```

Part B: Making Changes

14. **Add a new function**:

    Edit `sequence_analyzer.py`, add before `if __name__`:
    ```

```
    def reverse_complement(sequence):
        """Return the reverse complement of a DNA sequence."""
        complement = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}
        sequence = sequence.upper()
        rev_comp = ''.join([complement[base] for base in
sequence[::-1]])
        return rev_comp
```

15. **Check what changed**:

```
    git diff
```

    Question: What do the + and - symbols mean?

16. **Stage and commit**:

```
    git add sequence_analyzer.py
    git commit -m "Add reverse complement function"
```

17. **Add test in main block**:

    Modify the `if __name__` block:

```
    if __name__ == "__main__":
        test_seq = "ATGCATGCATGC"
        print(f"Original: {test_seq}")
        print(f"GC content: {gc_content(test_seq):.1f}%")
        print(f"Reverse complement: {reverse_complement(test_seq)}")
```

18. **View diff and commit**:

```
    git diff
    git add sequence_analyzer.py
    git commit -m "Add test for reverse complement"
```

19. **View history**:

```
    git log --oneline
```

Part C: Branches

20. **Create a feature branch**:

```
    git branch feature-translate
    git branch
```

    Question: Which branch are you currently on?

21. **Switch to the branch**:

```
    git switch feature-translate
    # or older syntax: git checkout feature-translate
    git branch
```

22. **Add translation function**:

    Add to `sequence_analyzer.py`:

```
def translate(sequence):
    """Translate DNA sequence to protein."""
    codon_table = {
        'ATG': 'M', 'TGG': 'W',
        'TTT': 'F', 'TTC': 'F',
        'TAA': '*', 'TAG': '*', 'TGA': '*',
        # Add more codons as needed
    }

    protein = []
    for i in range(0, len(sequence) - 2, 3):
        codon = sequence[i:i+3].upper()
        aa = codon_table.get(codon, 'X')  # X for unknown
        protein.append(aa)
        if aa == '*':  # Stop codon
            break

    return ''.join(protein)
```

23. **Commit the change**:

```
git add sequence_analyzer.py
git commit -m "Add basic translation function"
```

24. **Add test for translation**:

    Update `if __name__`:

```
if __name__ == "__main__":
    test_seq = "ATGTTTAA"
    print(f"Original: {test_seq}")
    print(f"GC content: {gc_content(test_seq):.1f}%")
    print(f"Reverse complement: {reverse_complement(test_seq)}")
    print(f"Translation: {translate(test_seq)}")
```

25. **Commit**:

```
git add sequence_analyzer.py
git commit -m "Add translation test"
```

26. **View branch history**:

```
git log --oneline --graph
```

27. **Switch back to main**:

```
git switch main
```

    Open `sequence_analyzer.py` - the `translate` function is gone!

Question: Why is the function missing?

28. **Merge the feature**:

```
git merge feature-translate
```

Open `sequence_analyzer.py` - now the function is there!

29. **View history graph**:

```
git log --oneline --graph --all
```

30. **Delete the feature branch**:

```
git branch -d feature-translate
git branch
```

Part D: Merge Conflicts

31. **Create two conflicting branches**:

```
git branch improve-gc
git branch improve-revcomp
```

32. **Make change in improve-gc**:

```
git switch improve-gc
```

Modify the `gc_content` function:

```
def gc_content(sequence):
    """Calculate GC content of DNA sequence."""
    sequence = sequence.upper()
    g_count = sequence.count('G')
    c_count = sequence.count('C')
    total = len(sequence)
    return (g_count + c_count) / total * 100
```

Commit:

```
git add sequence_analyzer.py
git commit -m "Improve GC content calculation clarity"
```

33. **Make different change in improve-revcomp**:

```
git switch improve-revcomp
```

Modify the `gc_content` function differently:

```
def gc_content(sequence):
    """Calculate GC content percentage of DNA sequence.

    Returns value between 0 and 100.
    """
```

```
            sequence = sequence.upper()
            gc_count = sequence.count('G') + sequence.count('C')
            return (gc_count / len(sequence)) * 100 if len(sequence) > 0
else 0
```

Commit:

```
git add sequence_analyzer.py
git commit -m "Add safety check for empty sequences"
```

34. **Merge first branch into main**:

```
git switch main
git merge improve-gc
```

This should merge without issues.

35. **Try to merge second branch**:

```
git merge improve-revcomp
```

Conflict! You should see:

```
CONFLICT (content): Merge conflict in sequence_analyzer.py
Automatic merge failed; fix conflicts and then commit the result.
```

36. **View the conflict**:

```
git status
```

Open `sequence_analyzer.py` and find:

```
<<<<<<< HEAD
def gc_content(sequence):
    """Calculate GC content of DNA sequence."""
    sequence = sequence.upper()
    g_count = sequence.count('G')
    c_count = sequence.count('C')
    total = len(sequence)
    return (g_count + c_count) / total * 100
=======
def gc_content(sequence):
    """Calculate GC content percentage of DNA sequence.

    Returns value between 0 and 100.
    """
    sequence = sequence.upper()
    gc_count = sequence.count('G') + sequence.count('C')
    return (gc_count / len(sequence)) * 100 if len(sequence) > 0
else 0
>>>>>>> improve-revcomp
```

37. **Resolve the conflict**:

Edit the function to combine both improvements:

```
def gc_content(sequence):
    """Calculate GC content percentage of DNA sequence.

    Returns value between 0 and 100.
    """
    if len(sequence) == 0:
        return 0

    sequence = sequence.upper()
    g_count = sequence.count('G')
    c_count = sequence.count('C')
    total = len(sequence)
    return (g_count + c_count) / total * 100
```

Remove all conflict markers (<<<<<<<, >>>>>>>).

38. **Complete the merge**:

```
git add sequence_analyzer.py
git status  # Should say "All conflicts fixed"
git commit -m "Merge improve-revcomp: combine both improvements"
```

39. **View final history**:

```
git log --oneline --graph --all
```

40. **Clean up branches**:

```
git branch -d improve-gc improve-revcomp
```

Questions to Answer:

1. What's the difference between `git add` and `git commit`?
2. Why use branches instead of committing directly to main?
3. What causes merge conflicts?
4. Can you prevent all merge conflicts? How?
5. What does `git log --oneline --graph --all` show?

Exercise 4: GitHub Workflows

Learning Objectives:

- Create GitHub repositories
- Push and pull changes
- Create and review pull requests
- Collaborate using forks
- Use GitHub features (Issues, Releases)

This exercise continues with the `gene_analysis` project from Exercise 3 (plain Python, no external packages). If you skipped Exercise 3, create a folder `gene_analysis`, run `git init`, and save the following file there as `sequence_analyzer.py` before you start (it is the state of the file at the end of Exercise 3, with the validation of Part D already merged); then run `git add .` and `git commit -m "Add sequence analyzer"` and also create the `README.md` shown in Exercise 3, step 9.

```
"""
DNA Sequence Analyzer
Simple tools for analyzing DNA sequences.
"""

def gc_content(sequence):
    """Calculate GC content percentage of DNA sequence.

    Returns value between 0 and 100.
    """
    if len(sequence) == 0:
        return 0

    sequence = sequence.upper()
    g_count = sequence.count('G')
    c_count = sequence.count('C')
    total = len(sequence)
    return (g_count + c_count) / total * 100


def reverse_complement(sequence):
    """Return the reverse complement of a DNA sequence."""
    complement = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}
    sequence = sequence.upper()
    rev_comp = ''.join([complement[base] for base in sequence[::-1]])
    return rev_comp


def translate(sequence):
    """Translate DNA sequence to protein."""
    codon_table = {
        'ATG': 'M', 'TGG': 'W',
        'TTT': 'F', 'TTC': 'F',
        'TAA': '*', 'TAG': '*', 'TGA': '*',
        # Add more codons as needed
    }

    protein = []
    for i in range(0, len(sequence) - 2, 3):
        codon = sequence[i:i+3].upper()
        aa = codon_table.get(codon, 'X')  # X for unknown
        protein.append(aa)
        if aa == '*':  # Stop codon
            break

    return ''.join(protein)
```

```
if __name__ == "__main__":
    test_seq = "ATGTTTTAA"
    print(f"Original: {test_seq}")
    print(f"GC content: {gc_content(test_seq):.1f}%")
    print(f"Reverse complement: {reverse_complement(test_seq)}")
    print(f"Translation: {translate(test_seq)}")
```

Before you start: you need a GitHub account, Git installed and a working GitHub login (see `resources/before_day3.md` and "Authenticating with GitHub" in the handout). If `git push` says `Authentication failed`, fix the login first.

Part A: Creating a GitHub Repository

1. **Go to GitHub**: https://github.com
2. **Create new repository**:
   - Click "New repository"
   - Name: `gene-analysis-tools`
   - Description: "Tools for analyzing DNA sequences"
   - Public
   - **Don't** initialize with README (we already have one)
   - Click "Create repository"
3. **Connect local repository**:

   Use the commands GitHub shows you:

   ```
   # In your gene_analysis directory
   git remote add origin https://github.com/YOUR_USERNAME/gene-analysis-tools.git
   git branch -M main
   git push -u origin main
   ```
4. **Verify on GitHub**:
   - Refresh your repository page
   - You should see all your files
   - Click through to view `README.md` and `sequence_analyzer.py`
5. **Add a better README on GitHub**:
   - Click `README.md`
   - Click pencil icon to edit
   - Add:

   ````
   ## Installation

   ```bash
   git clone https://github.com/YOUR_USERNAME/gene-analysis-tools.git
   cd gene-analysis-tools
   ````
   ````

Testing

```
python sequence_analyzer.py
```

```
- Scroll down
- Add commit message: "Improve README"
- Click "Commit changes"
```

6. **Pull changes to local**:

```
git pull origin main
```

Check `README.md` - it now has your changes!

Part B: Branch and Pull Request Workflow

7. **Create issue on GitHub**:

- Go to "Issues" tab
- Click "New issue"
- Title: "Add AT/GC ratio calculation"
- Comment: "Add function to calculate ratio of AT to GC content"
- Click "Submit new issue"
- Note the issue number (e.g., #1)

8. **Create feature branch locally**:

```
git switch -c add-at-gc-ratio
```

9. **Implement feature**:

Add to `sequence_analyzer.py`:

```
def at_gc_ratio(sequence):
    """Calculate ratio of AT content to GC content."""
    sequence = sequence.upper()
    at_count = sequence.count('A') + sequence.count('T')
    gc_count = sequence.count('G') + sequence.count('C')

    if gc_count == 0:
        return float('inf') if at_count > 0 else 0

    return at_count / gc_count
```

Add test:

```
if __name__ == "__main__":
    test_seq = "ATGTTTTAA"
    print(f"Original: {test_seq}")
    print(f"GC content: {gc_content(test_seq):.1f}%")
    print(f"AT/GC ratio: {at_gc_ratio(test_seq):.2f}")
```

```
    print(f"Reverse complement: {reverse_complement(test_seq)}")
    print(f"Translation: {translate(test_seq)}")
```

10. **Commit changes**:

```
git add sequence_analyzer.py
git commit -m "Add AT/GC ratio calculation

Implements #1: Calculate ratio of AT to GC content
Includes handling for edge cases (no GC content)"
```

Note: "Implements #1" links commit to issue!

11. **Push branch to GitHub**:

```
git push -u origin add-at-gc-ratio
```

12. **Create Pull Request on GitHub**:

- GitHub should show a banner "Compare & pull request"
- Click it (or go to "Pull requests" → "New pull request")
- Base: `main`, Compare: `add-at-gc-ratio`
- Title: "Add AT/GC ratio calculation"
- Description:

```
## Changes
- Added `at_gc_ratio()` function
- Handles edge cases (empty sequence, no GC)
- Added test in main block

## Testing
Tested with various sequences:
- Normal: ATGC... (works)
- AT-only: ATAT... (returns inf)
- Empty: "" (returns 0)

Closes #1
```

- Click "Create pull request"

13. **Review your own PR**:

- Click "Files changed" tab
- Hover over lines - you can comment
- Add a comment: "Should we add more comprehensive tests?"
- Click "Review changes" → "Approve" → "Submit review"

14. **Merge the PR**:

- Click "Merge pull request"
- Click "Confirm merge"
- Click "Delete branch" (on GitHub)

15. **Update local repository**:

```
git switch main
git pull origin main
git branch -d add-at-gc-ratio  # Delete local branch
```

16. **Check issue**:

    - Go to Issues tab
    - Issue #1 should be closed!
    - The PR is linked automatically

Part C: Collaboration Workflow

Note: You'll need a partner for this, or create a second GitHub account. If working alone, simulate both roles.

Person A (Repository Owner):

17. **Add collaborator**:
    - Go to repository Settings
    - Click "Collaborators"
    - Click "Add people"
    - Enter partner's GitHub username
    - Click "Add [username] to this repository"

Person B (Collaborator):

18. **Accept invitation**:

    - Check email or GitHub notifications
    - Accept collaboration invitation

19. **Clone repository**:

```
git clone https://github.com/PERSON_A_USERNAME/gene-analysis-tools.git
cd gene-analysis-tools
```

20. **Create feature branch**:

```
git switch -c add-sequence-validator
```

21. **Add validation function**:

    Add to `sequence_analyzer.py`:

```
def is_valid_dna(sequence):
    """Check if sequence contains only valid DNA bases."""
    valid_bases = set('ATGC')
    return all(base.upper() in valid_bases for base in sequence)

def validate_sequence(sequence):
    """Validate DNA sequence and raise error if invalid."""
    if not sequence:
```

```
            raise ValueError("Sequence cannot be empty")
        if not is_valid_dna(sequence):
            raise ValueError(f"Invalid DNA sequence: {sequence}")
        return True
```

22. **Update existing functions to use validation**:

    Modify `gc_content`:

```
def gc_content(sequence):
    """Calculate GC content percentage of DNA sequence.

    Returns value between 0 and 100.
    """
    validate_sequence(sequence)

    sequence = sequence.upper()
    g_count = sequence.count('G')
    c_count = sequence.count('C')
    total = len(sequence)
    return (g_count + c_count) / total * 100
```

23. **Commit and push**:

```
git add sequence_analyzer.py
git commit -m "Add sequence validation

- Add is_valid_dna() and validate_sequence()
- Update gc_content() to validate input
- Raises ValueError for invalid sequences"

git push -u origin add-sequence-validator
```

24. **Create Pull Request**:

    - Go to GitHub
    - Create PR: `add-sequence-validator` → `main`
    - Title: "Add DNA sequence validation"
    - Description: Explain the changes
    - Assign Person A as reviewer

Person A (Repository Owner):

25. **Review the PR**:
    - Go to "Pull requests"
    - Click on the PR from Person B
    - Click "Files changed"
    - Review the code
    - Click on a line number, add comment: "Should we also update the other functions to validate input?"
    - Click "Start a review"
    - Add a few more comments
    - Click "Review changes" → "Request changes" → "Submit review"

Person B (Collaborator):

26. **Address review comments**:

    Update other functions to use validation:

```
        def reverse_complement(sequence):
            """Return the reverse complement of a DNA sequence."""
            validate_sequence(sequence)
            complement = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}
            sequence = sequence.upper()
            rev_comp = ''.join([complement[base] for base in
sequence[::-1]])
            return rev_comp

        def translate(sequence):
            """Translate DNA sequence to protein."""
            validate_sequence(sequence)
            codon_table = {
                'ATG': 'M', 'TGG': 'W',
                'TTT': 'F', 'TTC': 'F',
                'TAA': '*', 'TAG': '*', 'TGA': '*',
                # Add more codons as needed
            }

            protein = []
            for i in range(0, len(sequence) - 2, 3):
                codon = sequence[i:i+3].upper()
                aa = codon_table.get(codon, 'X')  # X for unknown
                protein.append(aa)
                if aa == '*':  # Stop codon
                    break

            return ''.join(protein)

        def at_gc_ratio(sequence):
            """Calculate ratio of AT content to GC content."""
            validate_sequence(sequence)
            sequence = sequence.upper()
            at_count = sequence.count('A') + sequence.count('T')
            gc_count = sequence.count('G') + sequence.count('C')

            if gc_count == 0:
                return float('inf') if at_count > 0 else 0

            return at_count / gc_count
```

    Make sure `validate_sequence` is defined *above* the functions that call it (or at least before they are called in the `if __name__ == "__main__":` block at the bottom of the file).

27. **Commit and push updates**:

```
git add sequence_analyzer.py
git commit -m "Add validation to all functions

Addresses review comments"

git push
```

28. **Reply to review**:
    - Go back to PR on GitHub
    - Reply to review comments: "Updated all functions to validate input"

Person A (Repository Owner):

29. **Re-review and approve**:
    - Check the new commits
    - Click "Review changes" → "Approve" → "Submit review"
    - Click "Merge pull request"
    - Click "Confirm merge"

30. **Pull changes locally**:

```
git pull origin main
```

Part D: Fork Workflow

Simulating contributing to someone else's project:

31. **Fork a repository**:
    - Go to a public repository (or use a partner's)
    - Click "Fork" button (top right)
    - Click "Create fork"

32. **Clone YOUR fork**:

```
git clone https://github.com/YOUR_USERNAME/forked-repo.git
cd forked-repo
```

33. **Add upstream remote**:

```
git remote add upstream
https://github.com/ORIGINAL_OWNER/original-repo.git
git remote -v
```

    You should see:
    - `origin`: Your fork
    - `upstream`: Original repository

34. **Create feature branch**:

```
git switch -c fix-documentation
```

35. **Make changes**:

- Fix a typo in README
- Add more documentation
- Improve comments

36. **Commit and push to YOUR fork**:

```
git add .
git commit -m "Improve documentation"
git push -u origin fix-documentation
```

37. **Create Pull Request to original**:

- Go to YOUR fork on GitHub
- Click "Contribute" → "Open pull request"
- Base repository: original owner's repo
- Base: `main`
- Head repository: your fork
- Compare: `fix-documentation`
- Create pull request

38. **Keep fork updated** (do this regularly):

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

Part E: Releases and Documentation

39. **Create a release**:

- Go to repository main page
- Click "Releases" (right sidebar)
- Click "Create a new release"
- Click "Choose a tag"
- Type: `v1.0.0`
- Click "Create new tag"
- Release title: "Version 1.0.0 - Initial Release"
- Description:

````
## Features
- GC content calculation
- Reverse complement
- Translation
- AT/GC ratio
- Sequence validation

## Installation
```bash
git clone https://github.com/username/gene-analysis-tools.git
````
````

```
- Click "Publish release"
```

40. **View the release**:
    - Click on release
    - Note the zip/tar.gz download links
    - This provides a permanent snapshot

41. **Add topics to repository**:
    - Go to repository main page
    - Click gear icon next to "About"
    - Add topics: `dna`, `bioinformatics`, `python`, `genetics`
    - Add website (if you have one)
    - Save changes

42. **Update README with badges**:

    Add to top of README.md:

    ```
    # Gene Analysis Tools

    ![Python](https://img.shields.io/badge/python-3.8+-blue.svg)
    ![License](https://img.shields.io/badge/license-MIT-green.svg)
    ![Version](https://img.shields.io/badge/version-1.0.0-orange.svg)
    ```

    Commit and push.

Questions to Answer:

1. What's the difference between a fork and a clone?
2. When should you use a Pull Request vs direct commit?
3. Why link commits to issues (e.g., "Fixes #1")?
4. What are the benefits of code review?
5. How do releases differ from commits/tags?

Exercise 5: Integrated Workflow Project

Learning Objectives:

- Apply all tools in a complete workflow
- Create reproducible research project
- Collaborate effectively
- Document thoroughly

Project: Temperature Data Analysis

You'll create a complete analysis project with proper structure, version control, environments, and documentation. All the analysis code is written in plain Python (standard library only): data are generated with `random`, stored with the `csv` module and summarised with `statistics`. The only

external dependency is the tiny package `tabulate`, which is there to practise pinning a dependency in `requirements.txt`.

Part A: Project Setup

1. **Create project directory**:

   ```
   mkdir temperature_analysis
   cd temperature_analysis
   ```

2. **Initialize Git**:

   ```
   git init
   ```

3. **Create .gitignore**:

   ```
   # Python
   __pycache__/
   *.pyc
   .ipynb_checkpoints/

   # Environments
   venv/
   .venv/
   env/

   # Data (will be generated by the script)
   data/raw/

   # Results
   results/

   # IDE
   .vscode/
   .idea/
   ```

4. **Create project structure**:

   ```
   mkdir -p data/{raw,processed} notebooks src results
   touch README.md requirements.txt
   touch src/__init__.py
   touch notebooks/.gitkeep results/.gitkeep
   ```

   (On Windows, create the folders and empty files with your file explorer or editor.)

5. **Create README.md**:

   ```
   # Temperature Analysis Project

   Analysis of temperature trends over time, using plain Python.

   ## Project Structure
   ```

temperature_analysis/ ├── data/ │ ├── raw/ # Generated data (not in git) │ └── processed/ # Cleaned data ├── notebooks/ # Jupyter notebooks ├── src/ # Source code └── results/ # Analysis results

```
## Setup

### Create Environment
```bash
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Generate Data

```
python src/download_data.py
```

Usage

Process the data and run the analysis:

```
python src/data_processing.py
python -m src.analysis
```

Or run the analysis notebooks in order:

1. 01_data_exploration.ipynb
2. 02_statistical_analysis.ipynb
3. 03_decomposition.ipynb

Author

Your Name
```

6. **Commit initial structure**:

```
git add .
git commit -m "Initial project structure"
```

Part B: Environment Setup

7. **Create requirements.txt**:

```
tabulate==0.9.0
jupyterlab>=4.0.0
```

`tabulate` is a small pure-Python package that prints nicely formatted text tables. We pin its exact version (==) so that everybody uses the same one. `jupyterlab` is needed to run the notebooks inside this environment.

8. **Create virtual environment**:

```
python -m venv venv
source venv/bin/activate   # On Windows: venv\Scripts\activate
```

9. **Install dependencies**:

```
pip install -r requirements.txt
```

10. **Commit requirements**:

```
git add requirements.txt
git commit -m "Add project dependencies"
```

Part C: Data Acquisition

11. **Create data generation script**:

Create `src/download_data.py`. It simulates one year (day index 1 to 365) of daily temperatures with a fixed random seed, adds a few missing values and one duplicated record (like real data), and writes everything with the `csv` module:

```
"""
Generate sample temperature data for analysis.

The data are simulated with the standard library only, using a
fixed
random seed so that everybody gets exactly the same file.
"""
import csv
import math
import random
from pathlib import Path

N_DAYS = 365
FIELDNAMES = ['day', 'temperature', 'location']


def generate_sample_data(n_days=N_DAYS, seed=42):
    """
    Generate sample temperature data.

    Parameters
    ----------
    n_days : int
        Number of days to simulate (day index 1..n_days).
    seed : int
        Seed for the random number generator.

    Returns
    -------
    list of dict
```

```
            One dictionary per day with keys 'day', 'temperature',
'location'.
        """
        random.seed(seed)
        rows = []
        for day in range(1, n_days + 1):
            # Seasonal component: sine wave with a period of 365 days
            seasonal = 10 * math.sin(2 * math.pi * day / 365 - math.pi
/ 2)
            # Slight warming trend (0 to 2 degrees over the period)
            trend = 2 * (day - 1) / (n_days - 1)
            # Random measurement noise
            noise = random.gauss(0, 3)
            temperature = 18 + seasonal + trend + noise
            rows.append({
                'day': day,
                'temperature': round(temperature, 2),
                'location': 'City_A',
            })

        # Make the data a little bit messy, like real data:
        # a few missing measurements and one duplicated record
        for day in (50, 120, 200):
            rows[day - 1]['temperature'] = ''
        rows.append(dict(rows[99]))

        return rows


    def save_csv(rows, filepath):
        """Write a list of dictionaries to a CSV file."""
        with open(filepath, 'w', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=FIELDNAMES)
            writer.writeheader()
            writer.writerows(rows)


    def main():
        """Generate and save data."""
        print("Generating sample data...")
        rows = generate_sample_data()

        # Create data directory if needed
        data_dir = Path('data/raw')
        data_dir.mkdir(parents=True, exist_ok=True)

        # Save data
        output_file = data_dir / 'temperature_data.csv'
        save_csv(rows, output_file)
        print(f"Data saved to {output_file}")
        print(f"Generated {len(rows)} records")
```

```
if __name__ == "__main__":
    main()
```

12. **Run the script**:

    ```
    python src/download_data.py
    ```

    You should see `Generated 366 records` (365 days plus one duplicate). Open `data/raw/temperature_data.csv` in a text editor to look at the data.

13. **Commit the script** (but not the data!):

    ```
    git add src/download_data.py
    git commit -m "Add data generation script"
    ```

Part D: Data Processing Module

14. **Create processing module**:

    Create `src/data_processing.py`:

    ```
    """
    Data processing utilities (standard library only).
    """
    import csv
    import statistics
    from pathlib import Path

    PROCESSED_FIELDS = ['day', 'temperature', 'location', 'block',
                        'temp_7day_ma', 'temp_30day_ma']


    def load_raw_data(filepath):
        """
        Load raw temperature data.

        Parameters
        ----------
        filepath : str or Path
            Path to CSV file.

        Returns
        -------
        list of dict
            One dictionary per row. 'day' is an int, 'temperature' is
    a
            float (or None if the measurement is missing).
        """
        rows = []
        with open(filepath, newline='') as f:
    ```

```
                for record in csv.DictReader(f):
                    temperature = record['temperature']
                    rows.append({
                        'day': int(record['day']),
                        'temperature': float(temperature) if temperature
else None,
                        'location': record['location'],
                    })
            return rows


        def clean_data(rows):
            """
            Clean temperature data.

            Parameters
            ----------
            rows : list of dict
                Raw data.

            Returns
            -------
            list of dict
                Data without duplicates or missing values, sorted by day.
            """
            cleaned = []
            seen = set()
            for row in rows:
                # Remove missing values
                if row['temperature'] is None:
                    continue
                # Remove duplicates
                key = (row['day'], row['location'])
                if key in seen:
                    continue
                seen.add(key)
                cleaned.append(dict(row))

            # Sort by day
            cleaned.sort(key=lambda row: row['day'])
            return cleaned


        def moving_average(values, window):
            """
            Trailing moving average computed with a plain loop.

            The first (window - 1) positions have no full window and are
None.
            """
            result = []
```

```
            for i in range(len(values)):
                if i < window - 1:
                    result.append(None)
                else:
                    chunk = values[i - window + 1:i + 1]
                    result.append(sum(chunk) / window)
            return result


        def add_features(rows, block_size=30):
            """
            Add derived features.

            Parameters
            ----------
            rows : list of dict
                Cleaned data.
            block_size : int
                Number of days per block (a "month-like" period of 30
    days).

            Returns
            -------
            list of dict
                Data with the additional keys 'block', 'temp_7day_ma' and
                'temp_30day_ma'.
            """
            temperatures = [row['temperature'] for row in rows]
            ma7 = moving_average(temperatures, 7)
            ma30 = moving_average(temperatures, 30)

            for row, m7, m30 in zip(rows, ma7, ma30):
                row['block'] = (row['day'] - 1) // block_size + 1
                row['temp_7day_ma'] = m7
                row['temp_30day_ma'] = m30
            return rows


        def save_processed(rows, filepath):
            """Save processed rows to CSV (None becomes an empty
    field)."""
            filepath = Path(filepath)
            filepath.parent.mkdir(parents=True, exist_ok=True)
            with open(filepath, 'w', newline='') as f:
                writer = csv.DictWriter(f, fieldnames=PROCESSED_FIELDS)
                writer.writeheader()
                for row in rows:
                    writer.writerow({k: ('' if row[k] is None else row[k])
                                     for k in PROCESSED_FIELDS})
```

```
        def load_processed(filepath):
            """Load a processed CSV file written by save_processed()."""
            rows = []
            with open(filepath, newline='') as f:
                for record in csv.DictReader(f):
                    row = {'day': int(record['day']),
                           'location': record['location'],
                           'block': int(record['block'])}
                    for key in ('temperature', 'temp_7day_ma',
'temp_30day_ma'):
                        row[key] = float(record[key]) if record[key] else
None
                    rows.append(row)
            return rows


        def summarise(rows):
            """
            Summary statistics of the temperature column.

            Returns
            -------
            dict
                Keys: 'n', 'mean', 'median', 'stdev', 'min', 'max'.
            """
            values = [row['temperature'] for row in rows
                      if row['temperature'] is not None]
            return {
                'n': len(values),
                'mean': statistics.mean(values),
                'median': statistics.median(values),
                'stdev': statistics.stdev(values),
                'min': min(values),
                'max': max(values),
            }


        def process_data(input_path, output_path):
            """
            Complete data processing pipeline.

            Parameters
            ----------
            input_path : str or Path
                Path to raw data.
            output_path : str or Path
                Path to save processed data.

            Returns
            -------
            list of dict
```

```
        Processed data.
    """
    print("Loading data...")
    rows = load_raw_data(input_path)
    print(f"  {len(rows)} raw records")

    print("Cleaning data...")
    rows = clean_data(rows)
    print(f"  {len(rows)} records after cleaning")

    print("Adding features...")
    rows = add_features(rows)

    save_processed(rows, output_path)
    print(f"Processed data saved to {output_path}")

    return rows


if __name__ == "__main__":
    processed = process_data(
        'data/raw/temperature_data.csv',
        'data/processed/temperature_processed.csv'
    )
    stats = summarise(processed)
    print(f"Mean: {stats['mean']:.2f} C, "
          f"median: {stats['median']:.2f} C, "
          f"stdev: {stats['stdev']:.2f} C")
```

15. **Test the module**:

    ```
    python src/data_processing.py
    ```

    Expected output (the numbers are identical for everybody because the random seed is fixed):

    ```
    Loading data...
      366 raw records
    Cleaning data...
      362 records after cleaning
    Adding features...
    Processed data saved to data/processed/temperature_processed.csv
    Mean: 19.24 C, median: 18.99 C, stdev: 7.89 C
    ```

16. **Commit**:

    ```
    git add src/data_processing.py
    git commit -m "Add data processing module"
    ```

Part E: Analysis Notebooks

17. **Create the analysis module**:

Create `src/analysis.py`. It computes summaries for blocks of 30 days and a linear trend, and prints text tables and text bar charts:

```
        """
        Analysis functions (standard library plus tabulate).
        """
        import statistics

        from tabulate import tabulate


        def block_summary(rows, block_size=30):
            """
            Summarise the temperature in blocks of days.

            Parameters
            ----------
            rows : list of dict
                Cleaned data, each row with 'day' and 'temperature'.
            block_size : int
                Number of days per block.

            Returns
            -------
            list of dict
                One dictionary per block with 'block', 'first_day',
'last_day',
                'mean', 'min' and 'max'.
            """
            blocks = {}
            for row in rows:
                number = (row['day'] - 1) // block_size + 1
                blocks.setdefault(number, []).append(row)

            summary = []
            for number in sorted(blocks):
                members = blocks[number]
                values = [row['temperature'] for row in members]
                summary.append({
                    'block': number,
                    'first_day': members[0]['day'],
                    'last_day': members[-1]['day'],
                    'mean': statistics.mean(values),
                    'min': min(values),
                    'max': max(values),
                })
            return summary


        def linear_trend(rows):
            """
```

```
            Least-squares straight line through the temperature data.

            Returns
            -------
            tuple
                (slope, intercept) where slope is in degrees per day.
            """
            x = [row['day'] for row in rows]
            y = [row['temperature'] for row in rows]
            x_mean = sum(x) / len(x)
            y_mean = sum(y) / len(y)
            numerator = sum((xi - x_mean) * (yi - y_mean) for xi, yi in
zip(x, y))
            denominator = sum((xi - x_mean) ** 2 for xi in x)
            slope = numerator / denominator
            intercept = y_mean - slope * x_mean
            return slope, intercept


        def text_bar_chart(labels, values, width=40):
            """
            Build a simple horizontal bar chart made of '#' characters.

            Bars are scaled so that the largest value has `width`
characters.
            """
            largest = max(values)
            lines = []
            for label, value in zip(labels, values):
                bar = '#' * round(value / largest * width) if largest > 0
else ''
                lines.append(f"{label:>10} | {bar} {value:.1f}")
            return '\n'.join(lines)


        def table_text(rows, headers):
            """Format rows (a list of lists) as a plain-text table."""
            return tabulate(rows, headers=headers, floatfmt=".2f")


        def html_table(rows, headers):
            """Format rows (a list of lists) as an HTML table string."""
            html = ['<table>', '<tr>']
            html += [f'<th>{h}</th>' for h in headers]
            html.append('</tr>')
            for row in rows:
                html.append('<tr>')
                for value in row:
                    text = f'{value:.2f}' if isinstance(value, float) else
value
                    html.append(f'<td>{text}</td>')
```

```
            html.append('</tr>')
        html.append('</table>')
        return ''.join(html)


if __name__ == "__main__":
    # Run from the project root with:  python -m src.analysis
    from src.data_processing import load_processed

    data =
load_processed('data/processed/temperature_processed.csv')
    summary = block_summary(data)

    table = [[s['block'], s['first_day'], s['last_day'],
              s['mean'], s['min'], s['max']] for s in summary]
    print(table_text(table, ['Block', 'From', 'To', 'Mean', 'Min',
'Max']))
    print()
    print(text_bar_chart([f"block {s['block']}" for s in summary],
                         [s['mean'] for s in summary]))
    slope, intercept = linear_trend(data)
    print(f"\nOverall trend: {slope * 365:.2f} C per 365 days")
```

Try it from the project root:

```
python -m src.analysis
```

Commit it:

```
git add src/analysis.py
git commit -m "Add analysis module"
```

18. **Start JupyterLab**:

    ```
    jupyter lab
    ```

19. **Create exploration notebook**:

    Create `notebooks/01_data_exploration.ipynb`:

    Cell 1 (Markdown):

    ```
    # Temperature Data Exploration

    Initial exploration of temperature data.

    **Author**: Your Name
    **Date**: YYYY-MM-DD
    ```

    Cell 2 (Code - Setup):

    ```
    import sys
    sys.path.append('../')
    ```

```
from IPython.display import display, Markdown, HTML
from src.data_processing import load_raw_data, summarise
from src.analysis import html_table, text_bar_chart
```

Cell 3 (Code - Load):

```
# Load raw data
rows = load_raw_data('../data/raw/temperature_data.csv')
print(f"Loaded {len(rows)} records")

# Show the first five rows as an HTML table
first = [[r['day'], r['temperature'], r['location']] for r in
rows[:5]]
display(HTML(html_table(first, ['Day', 'Temperature',
'Location'])))
```

Cell 4 (Code - Summary):

```
# Summary statistics
stats = summarise(rows)
display(Markdown(
    f"**Records with a measurement**: {stats['n']}  \n"
    f"**Mean**: {stats['mean']:.2f} °C  \n"
    f"**Median**: {stats['median']:.2f} °C  \n"
    f"**Standard deviation**: {stats['stdev']:.2f} °C  \n"
    f"**Range**: {stats['min']:.1f} to {stats['max']:.1f} °C"
))
```

Cell 5 (Code - First days as text bars):

```
# The first 20 days (skipping missing values)
first_days = [r for r in rows[:20] if r['temperature'] is not
None]
print(text_bar_chart([f"day {r['day']}" for r in first_days],
                     [r['temperature'] for r in first_days]))
```

Cell 6 (Code - Distribution):

```
# Distribution: count the measurements in 5-degree bins
bin_width = 5
counts = {}
for r in rows:
    if r['temperature'] is not None:
        low = int(r['temperature'] // bin_width) * bin_width
        counts[low] = counts.get(low, 0) + 1

lows = sorted(counts)
print(text_bar_chart([f"{low} to {low + bin_width}" for low in
lows],
                     [counts[low] for low in lows]))
```

Cell 7 (Markdown):

```
## Observations

- Data shows a clear seasonal pattern
- Some random fluctuations visible
- The distribution is wide and fairly flat (it mixes cold and warm
seasons)
- Mean temperature around 19°C
```

20. **Process data and create analysis notebook**:

    Create `notebooks/02_statistical_analysis.ipynb`:

    Cell 1 (Markdown):

```
# Statistical Analysis

Detailed statistical analysis of temperature trends.
```

    Cell 2 (Code - Setup):

```
import sys
sys.path.append('../')

from IPython.display import display, Markdown, HTML
from src.data_processing import process_data
from src.analysis import (block_summary, linear_trend, html_table,
                          text_bar_chart)
```

    Cell 3 (Code - Process data):

```
# Process data
rows = process_data(
    '../data/raw/temperature_data.csv',
    '../data/processed/temperature_processed.csv'
)

# Moving averages around day 30 (the first 29 values of the 30-day
average are empty)
display(HTML(html_table(
    [[r['day'], r['temperature'], r['temp_7day_ma'],
r['temp_30day_ma']]
     for r in rows[28:34]],
    ['Day', 'Temperature', '7-day MA', '30-day MA'])))
```

    Cell 4 (Code - Block analysis):

```
# Statistics for blocks of 30 days
summary = block_summary(rows, block_size=30)
table = [[s['block'], s['first_day'], s['last_day'],
          s['mean'], s['min'], s['max']] for s in summary]
display(HTML(html_table(table, ['Block', 'From', 'To', 'Mean',
'Min', 'Max'])))
```

Cell 5 (Code - Trend analysis):

```
# Linear trend
slope, intercept = linear_trend(rows)
display(Markdown(
    f"**Linear trend**: {slope:.4f} °C/day "
    f"({slope * 365:.2f} °C per 365 days)"
))
```

Cell 6 (Code - Text bar chart of block means):

```
print(text_bar_chart([f"block {s['block']}" for s in summary],
                     [s['mean'] for s in summary]))
```

21. **Save notebooks and commit**:

```
git add notebooks/
git commit -m "Add exploration and analysis notebooks"
```

Part F: Create GitHub Repository

22. **Create repository on GitHub**:

- Name: `temperature-analysis`
- Description: "Statistical analysis of temperature trends"
- Public
- Don't initialize

23. **Connect and push**:

```
git remote add origin
https://github.com/YOUR_USERNAME/temperature-analysis.git
git branch -M main
git push -u origin main
```

24. **Add LICENSE**:

- On GitHub, click "Add file" → "Create new file"
- Name: `LICENSE`
- Click "Choose a license template"
- Select "MIT License"
- Fill in year and name
- Commit

25. **Pull license locally**:

```
git pull origin main
```

Part G: Collaborative Feature

26. **Create issue**:

- On GitHub: Issues → New issue

- Title: "Add trend and residual decomposition"
- Body: "Split the temperature series into a slow trend (moving average) and residual fluctuations"
- Submit

27. **Create feature branch**:

```
git switch -c trend-decomposition
```

28. **Add analysis function**:

Add to `src/analysis.py`, before the `if __name__ == "__main__":` block:

```
def decompose(values, window=31):
    """
    Split a series into a slow trend and fast residual
fluctuations.

    The trend is a centred moving average over `window` points
    (`window` must be odd). The first and last window // 2
positions
    have no full window, so trend and residual are None there.

    Parameters
    ----------
    values : list of float
        Time series data.
    window : int
        Odd number of points in the moving average.

    Returns
    -------
    dict
        Dictionary with 'trend' and 'residual' lists (same length
as
        `values`).
    """
    half = window // 2
    trend = []
    residual = []
    for i, value in enumerate(values):
        if i < half or i >= len(values) - half:
            trend.append(None)
            residual.append(None)
        else:
            chunk = values[i - half:i + half + 1]
            smooth = sum(chunk) / window
            trend.append(smooth)
            residual.append(value - smooth)
    return {'trend': trend, 'residual': residual}
```

29. **Create notebook to test**:

Create `notebooks/03_decomposition.ipynb`:

```
# Setup
import sys
sys.path.append('../')

import statistics
from IPython.display import display, Markdown
from src.data_processing import load_processed
from src.analysis import decompose, text_bar_chart

# Load data
rows =
load_processed('../data/processed/temperature_processed.csv')
temperatures = [r['temperature'] for r in rows]

# Decompose
components = decompose(temperatures, window=31)

residuals = [x for x in components['residual'] if x is not None]
display(Markdown(
    f"Residual standard deviation: **
{statistics.stdev(residuals):.2f} °C**"
))

# Trend sampled every 30 days
labels, values = [], []
for i in range(0, len(rows), 30):
    if components['trend'][i] is not None:
        labels.append(f"day {rows[i]['day']}")
        values.append(components['trend'][i])
print(text_bar_chart(labels, values))
```

30. **Commit and push**:

```
git add src/analysis.py notebooks/03_decomposition.ipynb
git commit -m "Add trend and residual decomposition

Implements #1: Split temperature into trend and residual"
git push -u origin trend-decomposition
```

31. **Create Pull Request**:

- On GitHub: Create PR
- Title: "Add trend and residual decomposition"
- Description: Link to issue, explain changes
- Create PR

32. **Merge (if working alone, or wait for review)**:

- Review files changed
- Merge pull request
- Delete branch on GitHub

33. **Update local**:

```
git switch main
git pull origin main
git branch -d trend-decomposition
```

Part H: Finalize and Release

34. **Update README with results**:

Add to README.md:

```
## Results

### Key Findings
- Clear seasonal pattern with a 365-day period (warmest around day
180)
- Slight warming trend of about 1.6°C per 365 days (fitted line)
- Random variations of about ±3°C around the slow trend

### Example output

Mean temperature per 30-day block (`python -m src.analysis`):

    block 1 | ############ 8.6
    block 2 | ############### 11.3
    block 3 | ##################### 15.7
    ...

## Reproducibility

All analyses are fully reproducible:
1. Clone repository
2. Set up environment
3. Run data generation script
4. Run `src/data_processing.py` and the notebooks in order
```

35. **Create final summary notebook**:

notebooks/00_summary.ipynb:

```
# Temperature Analysis: Summary

## Project Overview
Analysis of temperature trends over one year (day 1 to day 365).

## Methodology
1. Data generation/collection
2. Data cleaning and feature engineering
3. Statistical analysis
4. Trend and residual decomposition
```

```
## Key Results
- **Trend**: about +1.6°C per 365 days
- **Seasonality**: Clear 365-day cycle
- **Variability**: ±3°C random fluctuations

## Notebooks
- `01_data_exploration.ipynb`: Initial exploration
- `02_statistical_analysis.ipynb`: Trend analysis
- `03_decomposition.ipynb`: Trend and residual components
```

36. **Commit everything**:

```
git add .
git commit -m "Final updates: README and summary"
git push origin main
```

37. **Create release**:

- GitHub: Releases → Create new release
- Tag: `v1.0.0`
- Title: "Version 1.0.0 - Complete Analysis"
- Description:

```
## Complete temperature analysis package

### Features
- Data generation and processing
- Block summaries and linear trend
- Trend and residual decomposition
- Text tables and text bar charts
- Fully reproducible workflow

### How to Use
See README.md for complete instructions
```

- Publish release

38. **Add DOI (optional)**:

- Go to Zenodo.org
- Connect to GitHub
- Enable repository
- Release automatically archived

Part I: Documentation Review

39. **Create checklist**:

Create `CHECKLIST.md`:

```
# Reproducibility Checklist

- [x] Code in version control (Git/GitHub)
```

```
- [x] Dependencies documented (requirements.txt)
- [x] Data generation documented (script)
- [x] Analysis steps documented (notebooks)
- [x] Random seeds set (in data generation script)
- [x] README explains workflow
- [x] Results can be reproduced
- [x] Tables and charts generated from code
- [x] Environment can be recreated
- [x] Licensed (MIT)
- [x] Released (v1.0.0)
```

40. **Test full reproducibility**:

    In a new directory:

```
git clone https://github.com/YOUR_USERNAME/temperature-
analysis.git
cd temperature-analysis
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python src/download_data.py
python src/data_processing.py
python -m src.analysis
jupyter lab
# Run all notebooks
```

Questions to Reflect On:

1. How does version control improve research reproducibility?
2. What are the benefits of modular code (separate modules vs all in notebooks)?
3. How does documentation help your future self and collaborators?
4. What would you do differently in your next project?
5. How can you apply these practices to your research?

Additional Challenges

Challenge 1: Command-Line Arguments

Make `src/download_data.py` accept the number of days and the random seed from the command line using `sys.argv` (for example `python src/download_data.py 200 7`), with the current values as defaults.

Challenge 2: Input Validation

Add input validation with `try/except` to `load_raw_data()`: report a clear error message if the file does not exist, and skip (and count) rows whose temperature cannot be converted to a number or lies outside a plausible range.

Challenge 3: A Second Summary

Add a function that finds the warmest and the coldest 30-day block and the standard deviation within each block, print it as a table, and save it to `results/block_summary.csv` with the `csv` module.

Challenge 4: Improve Documentation

Add examples with expected output to the README, make sure every function has a complete docstring, and add a `CHANGELOG.md` that describes what changed in each release.

Challenge 5: Add a Pinned Dependency

Add `requests==2.31.0` to `requirements.txt`, write a small script that uses it to download a plain CSV file from a URL of your choice into `data/raw/`, and document the new step in the README. Recreate your environment from scratch to check that everything still works.

Evaluation Criteria

For each exercise, you should be able to:

1. **Execute successfully**: All commands work without errors
2. **Understand**: Explain what each step does
3. **Troubleshoot**: Debug common issues
4. **Adapt**: Apply concepts to different scenarios
5. **Teach**: Explain concepts to others

Getting Help

If stuck:

1. **Read error messages carefully**: They often tell you what's wrong
2. **Check documentation**: Official docs are authoritative
3. **Search online**: Stack Overflow, GitHub issues
4. **Ask specific questions**: "This command gives this error" vs "It doesn't work"
5. **Use `--help`**: Most commands have built-in help
6. **Check Git status often**: `git status` tells you what's happening

---

Next Steps

After completing these exercises:

1. **Apply to your research**: Implement in your own projects
2. **Practice regularly**: Make it part of your workflow
3. **Share knowledge**: Help others learn

4. **Stay current**: Tools evolve, keep learning
5. **Contribute**: Contribute to open source projects

> **Good luck with the exercises! Remember: programming is learned by doing. Don't be afraid to experiment, make mistakes, and ask questions.**

Document Version: 1.0
Estimated Total Time: 6-8 hours